SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

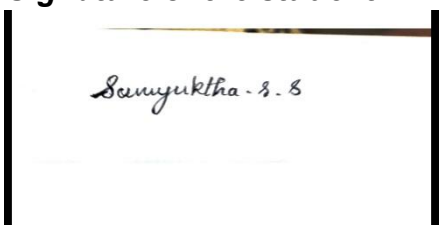
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I Samyuktha S S(192311056) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS

Question 1: Errors in Equivalence Class Partitioning and Boundary Value Analysis

1. Introduction

Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) are black-box testing techniques used to identify defects in input validation. In an online shopping system, incorrect grouping of valid and invalid inputs and failure to test boundaries can leave important defects undetected.

2. Errors in the Test Design

The tester has made two major mistakes:

1. Incorrect equivalence classes: Valid and invalid inputs are grouped together, even though they may produce different system behavior.
2. Missing boundary tests: Values at the minimum and maximum limits and just outside those limits are not tested.

For example, if the permitted quantity is 1–100, grouping negative, zero, valid, and excessive quantities into one class is incorrect.

3. Root Causes

- Lack of understanding of input validation rules.
- Failure to identify valid and invalid input ranges.
- Inadequate analysis of minimum and maximum values.
- Selecting test cases without considering likely boundary defects.

4. Corrected Equivalence Classes

Assume an online shopping system allows quantities from 1 to 100.

Equivalence class	Example input	Expected result
Negative quantity	-1	Reject
Zero quantity	0	Reject
Valid quantity	1–100	Accept if stock is available
Excessive quantity	101 or more	Reject

5. Boundary Value Analysis

Test value	Expected result
0	Reject

1	Accept if stock is available
2	Accept if stock is available
99	Accept if stock is available
100	Accept if stock is available
101	Reject

These values assume the allowed range is 1–100. Actual system limits should be used when available.

6. Improved Testing Approach

The tester should first identify the input rules, divide inputs into valid and invalid equivalence classes, and select representative values from each class. BVA should then test values at and around the limits. Separate tests should also verify product availability, invalid prices, and incomplete order details.

Conclusion

Correct ECP and BVA improve test coverage by examining representative inputs and boundary conditions. This helps detect validation errors, prevent invalid orders, and improve the reliability of the online shopping system.

Question 2: Errors in Random Testing of a Banking Application

1. Introduction

Random testing selects inputs without a structured selection strategy. Although it can expose unexpected defects, it does not guarantee adequate coverage of important transaction conditions. In a banking application, missing transaction limits and boundary conditions can lead to incorrect financial operations.

2. Errors in the Testing Strategy

- Test inputs are selected without systematic planning.
- Minimum and maximum transaction limits may not be tested.
- Important combinations of balance, amount, and account status may be missed.
- High test counts are incorrectly treated as proof of high coverage.
- Expected results and coverage criteria may not be clearly defined.

3. Root Cause Analysis

Random testing may generate many ordinary values but rarely select the exact values needed to expose boundary defects. Without equivalence classes, boundary analysis, and requirements-based test cases, critical scenarios may remain untested.

For example, a system accepting transfers from ₹100 to ₹50,000 may work correctly for ₹1,000 but fail at ₹100 or ₹50,001.

4. Recommended Black-Box Techniques

Equivalence Class Partitioning (ECP): Divide inputs into valid and invalid groups, such as valid transaction amounts, zero amounts, negative amounts, and amounts above the permitted limit.

Boundary Value Analysis (BVA): Test values at and around the limits. For a range of ₹100–₹50,000, test ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001.

Decision Table Testing: Test combinations such as sufficient balance, valid amount, active account, and permitted transaction status.

State Transition Testing: Verify transaction behavior as it moves through states such as initiated, pending, successful, and failed, where applicable.

5. Improved Testing Strategy

1. Define the banking application's transaction rules.
2. Identify valid and invalid input classes.
3. Test all important boundary values.
4. Test insufficient balance and inactive accounts.
5. Verify transaction failure and recovery behavior.
6. Use random testing as an additional technique after structured tests.
7. Record results and measure coverage against defined criteria.

Conclusion

Random testing alone cannot guarantee that critical transaction scenarios are covered. Combining ECP, BVA, decision tables, and state transition testing provides systematic coverage, improves defect detection, and increases banking application reliability.

Question 3: Errors in Control Flow Graph Construction and Path Testing

1. Introduction

White-box testing examines the internal structure of software. A Control Flow Graph (CFG) represents the execution flow of a program, including statements, decisions, loops, and branches. If a developer omits decision nodes or loops, the resulting graph will be incomplete and path coverage will be inadequate.

2. Errors in the CFG

The main errors include:

1. Missing decision nodes for if and else conditions.
2. Omitting loop conditions and loop-back edges.

3. Failing to represent alternative execution paths.
4. Identifying too few independent paths.
5. Designing test cases that exercise only the normal execution path.

3. Root Causes

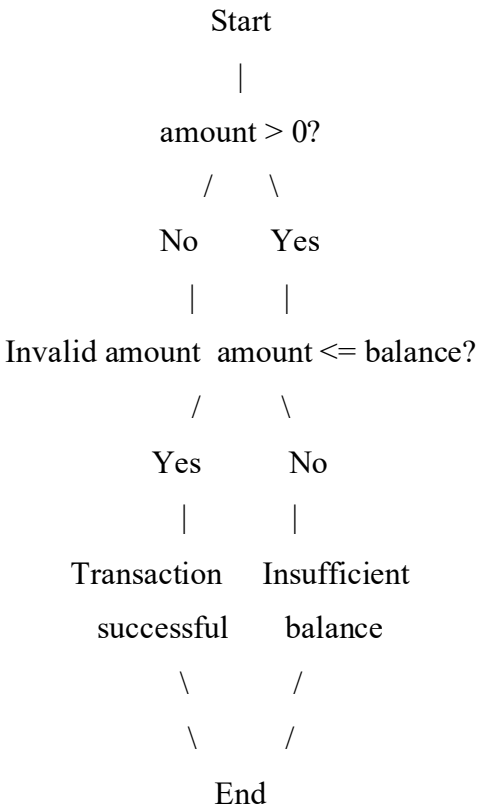
These errors may occur because the developer focuses only on statements rather than control structures. Inadequate code analysis and the absence of coverage measurements can also result in missing branches and loops.

4. Corrected Control Flow Representation

Consider this illustrative program:

if amount > 0: if amount <= balance: print("Transaction successful") else: print("Insufficient balance")else: print("Invalid amount")

Its conceptual CFG is:



5. Independent Execution Paths

The program contains three principal paths:

- Path 1: Positive amount → Amount does not exceed balance → Transaction successful.
- Path 2: Positive amount → Amount exceeds balance → Insufficient balance.
- Path 3: Amount is zero or negative → Invalid amount.

6. Corrected Test Cases

Test case	Input condition	Expected output
TC1	Amount = ₹500, balance = ₹1,000	Transaction successful
TC2	Amount = ₹1,500, balance = ₹1,000	Insufficient balance
TC3	Amount = ₹0	Invalid amount
TC4	Amount = -₹100	Invalid amount

These tests exercise all three principal paths in the example.

7. Improved Path Testing

The tester should inspect the complete source code, construct the CFG, identify decision and loop nodes, and determine the independent paths. Test cases should be designed to execute each feasible path. Statement, branch, and condition coverage should be measured separately.

For a single connected CFG, cyclomatic complexity can be calculated using:

$$V(G)=E-N+2$$

where E is the number of edges and N is the number of nodes.

Conclusion

A complete CFG is essential for effective white-box testing. Including all decisions, branches, and loops helps identify independent paths and develop stronger test cases. This improves coverage and increases the likelihood of detecting logical defects.

Question 4: Why 100% Statement Coverage Is Insufficient

1. Introduction

Statement coverage measures whether every executable statement has run at least once. However, achieving 100% statement coverage does not guarantee that every decision outcome has been tested. Therefore, logical errors in conditional statements may remain undetected.

2. Errors in the Existing Testing

The testing team has relied only on statement coverage. This approach may miss:

- False outcomes of conditional statements.
- Incorrect Boolean expressions.
- Errors in if-else branches.
- Defects involving combinations of conditions.
- Paths that execute only under particular inputs.

3. Example

Consider the following code:

```
if age >= 18:    print("Eligible")else:    print("Not eligible")
```

If the tester uses only age = 25, the statement print("Eligible") executes, but the else branch remains untested. Thus, statement coverage is incomplete in this example.

To achieve full statement coverage, both branches must execute at least once. Even then, more complex conditions may require additional tests.

4. Improved Coverage Criteria

Branch Coverage: Ensures each decision outcome is executed at least once. For the example, test one age below 18 and one age of 18 or above.

Condition Coverage: Ensures each individual Boolean condition evaluates to both True and False.

Path Coverage: Tests independent execution paths through the program. Complete path coverage may be impractical when loops create many possible paths.

Boundary Value Analysis: Tests values at and around critical limits, such as ages 17, 18, and 19.

5. Corrected Test Cases

Test case	Age	Expected result
TC1	17	Not eligible
TC2	18	Eligible
TC3	19	Eligible

These tests cover both branches and the important boundary.

6. Why the Revised Approach Is Better

Branch coverage detects untested decision outcomes, condition coverage helps expose Boolean logic defects, and path testing examines alternative execution routes. BVA complements these techniques by testing values around input limits.

Conclusion

100% statement coverage alone is not sufficient to prove software correctness. Combining statement, branch, condition, and path coverage provides a stronger testing strategy and improves the detection of logical errors.

Question 5: Incorrect Equivalence Class Partitioning for Age Validation

1. Introduction

Equivalence Class Partitioning divides input data into groups that are expected to behave similarly. In an age-validation field accepting values from 0 to 100, defining only two classes—negative and positive numbers—is incorrect because it fails to distinguish valid ages from positive values outside the permitted range.

2. Errors in the Existing Partitioning

The tester has made these mistakes:

1. All positive values are grouped together, including valid ages and values above 100.
2. The age value 0 is not clearly classified.
3. Values greater than 100 are not identified as invalid.
4. Boundary Value Analysis is not applied.
5. The test design does not cover the exact minimum and maximum limits.

3. Root Causes

The tester has not considered the complete validation requirement. The partitioning is based only on whether a number is positive or negative rather than whether it falls within the permitted age range. Failure to analyze boundary values further reduces defect detection.

4. Corrected Equivalence Classes

Assuming the allowed age range is inclusive, from 0 to 100:

Equivalence class	Example input	Expected result
Negative age	-1	Reject
Valid age	0–100	Accept
Age above maximum	101 or more	Reject
Non-numeric input	"abc"	Reject
Missing input	Empty field	Reject if age is mandatory

5. Boundary Value Analysis

Test value	Expected result
-1	Reject
0	Accept
1	Accept
99	Accept
100	Accept
101	Reject

The non-numeric and empty-input cases should also be tested because they represent invalid input formats.

6. Improved Testing Strategy

1. Confirm the permitted age range.
2. Separate valid and invalid equivalence classes.
3. Select representative values from each class.
4. Test values at and around the minimum and maximum limits.
5. Test non-numeric and missing inputs.

6. Verify that invalid ages produce suitable validation messages.
7. Repeat the tests after any validation changes.

Conclusion

Correct ECP separates valid ages from negative values and values exceeding the maximum limit. Combining it with BVA ensures that the boundaries 0 and 100 are tested thoroughly. This improves input validation, prevents invalid age values from being accepted, and increases the reliability of the application.

Question 6: Errors in Condition Coverage

1. Introduction

Condition coverage verifies whether each individual Boolean condition in a decision evaluates to both True and False. A developer who claims full condition coverage without testing both outcomes of every condition may leave logical defects undetected.

2. Errors in the Testing Approach

- Individual conditions may not have been tested for both True and False outcomes.
- Testing the overall decision result alone may overlook defects in individual conditions.
- Important combinations of Boolean expressions may be missed.
- The test suite may incorrectly claim full coverage without sufficient evidence.

3. Root Causes

These problems occur when testers do not distinguish condition coverage from statement or branch coverage. They may test only normal inputs and overlook combinations that affect the overall decision.

4. Example

Consider the condition:

if A and B: print("Accepted")else: print("Rejected")

A suitable test set is:

Test	A	B	Expected result
TC1	True	True	Accepted
TC2	True	False	Rejected
TC3	False	True	Rejected

Here, A evaluates to both True and False, and B also evaluates to both True and False. Therefore, both individual conditions achieve condition coverage.

5. Improved Testing Strategy

1. Identify every Boolean condition in the decision.

2. Select inputs that make each condition True and False.
3. Verify the overall decision result.
4. Measure condition and branch coverage separately.
5. Add tests for important combinations of conditions.

Conclusion

Condition coverage helps reveal defects hidden within compound Boolean expressions. A structured test set that exercises both outcomes of every condition provides better evidence of testing adequacy.

Question 7: Errors in Decision Table Testing for a Discount System

1. Introduction

Decision Table Testing verifies the behavior of a system under different combinations of conditions. In a discount system, the conditions may include premium membership and coupon usage. Missing combinations can cause incorrect price calculations.

2. Errors in the Existing Decision Table

- Not testing every combination of premium membership and coupon usage.
- Missing rules for customers who are not premium members.
- Failing to test customers who use a coupon without premium membership.
- Not defining whether discounts can be combined.
- Failing to verify the final price against expected results.

3. Root Causes

The testing team may have considered only common scenarios instead of all possible condition combinations. Unclear discount requirements and incomplete decision-table construction can also cause gaps.

4. Corrected Decision Table

Assume premium members receive a 10% discount and a valid coupon provides a 5% discount. For this example, assume both discounts can be combined additively.

Rule	Premium member?	Coupon applied?	Expected discount
R1	Yes	Yes	15%
R2	Yes	No	10%
R3	No	Yes	5%
R4	No	No	0%

These percentages and the combination rule are illustrative; the actual business requirements must determine the expected discount.

5. Improved Test Cases

1. Premium member uses a valid coupon.
2. Premium member does not use a coupon.
3. Non-premium member uses a valid coupon.
4. Non-premium member does not use a coupon.
5. An invalid or expired coupon is entered.
6. Verify that the final price is calculated correctly.
7. Check that the same coupon cannot be applied repeatedly if prohibited.

Conclusion

A complete decision table covers all combinations of relevant conditions. Testing each rule helps detect pricing errors, ensures consistent discount application, and reduces production defects.

Question 8: Errors in Loop Testing

1. Introduction

Loop Testing verifies the correctness of loops by examining their behavior for different iteration counts. Testing only the first and maximum iterations is inadequate because defects may occur when the loop executes zero times or exceeds its permitted limit.

2. Errors in the Existing Strategy

- The zero-iteration case is omitted.
- The case immediately beyond the maximum is not tested.
- Incorrect loop termination conditions may remain undetected.
- Nested loops may not be tested for interactions between their iteration counts.

3. Root Causes

The tester has selected only normal loop conditions and ignored boundary cases. This can miss off-by-one errors, infinite loops, incorrect termination, and problems caused by empty input collections.

4. Corrected Loop Testing Strategy

For a loop with a maximum of \(n\) iterations, test:

- Zero iterations: \(n=0\), where permitted.
- One iteration: \(n=1\).
- Two iterations: \(n=2\).
- Typical iterations: A representative normal value.
- Maximum iterations: \(n=\text{max}\).
- Beyond maximum: \(n=\text{max}+1\), to verify appropriate rejection or handling.

5. Nested Loop Testing

For nested loops, test the inner and outer loops separately before combining them.

1. Set the outer loop to zero iterations where possible.
2. Test the inner loop with zero, one, and multiple iterations.
3. Test normal combinations of outer and inner iterations.
4. Test their permitted maximums.
5. Check termination and output correctness.
6. Monitor performance for unexpectedly long execution.

6. Example Test Cases

Test case	Iteration count	Purpose
TC1	0	Verify zero-iteration handling
TC2	1	Verify single execution
TC3	2	Verify repeated execution
TC4	Maximum	Verify the upper limit
TC5	Maximum + 1	Verify limit enforcement

Conclusion

A proper Loop Testing strategy examines zero, minimum, normal, maximum, and beyond-maximum conditions. Testing nested loops systematically helps detect termination errors, boundary defects, and unexpected execution behavior.

Question 9: Errors in Using Error Guessing for a Safety-Critical Braking System

1. Introduction

Error Guessing relies on a tester's experience to predict likely defects. Although useful, it is insufficient as the primary technique for a safety-critical braking system because missed defects can result in serious accidents.

2. Errors in the Existing Approach

- Testing relies heavily on personal experience rather than systematic coverage.
- Critical braking thresholds may not be tested thoroughly.
- Different braking states and transitions may be overlooked.
- There may be no measurable coverage criteria.
- Unexpected failures and recovery conditions may remain untested.

3. Root Causes

The main causes are overreliance on ad-hoc testing, insufficient risk analysis, and failure to apply structured techniques. Error Guessing cannot guarantee that all important input boundaries and system-state transitions have been examined.

4. Applying Boundary Value Analysis

Suppose a braking sensor accepts readings from 0 to 100 units. These values are illustrative and do not represent real braking-system specifications.

Sensor reading	Expected test behavior
-1	Reject as invalid
0	Verify minimum-boundary behavior
1	Verify behavior just above minimum
99	Verify behavior just below maximum
100	Verify maximum-boundary behavior
101	Reject as invalid

Actual expected responses must be based on the safety requirements.

5. Applying State Transition Testing

Possible system states include:

- Idle: No braking request is active.
- Braking: A braking request is being processed.
- Emergency Braking: An emergency condition is detected.
- Fault: A sensor or system failure is detected.

Tests should verify valid transitions, such as Idle to Braking, and appropriate fault handling when sensor inputs are invalid or a component fails.

6. Improved Testing Strategy

1. Identify safety-critical requirements and risks.
2. Apply BVA to sensor inputs and braking thresholds.
3. Use State Transition Testing for braking and fault states.
4. Test sensor failures and unexpected input values.
5. Verify that fault conditions trigger the required safe response.
6. Use Error Guessing as a supplementary technique.
7. Maintain documented test results and traceability to requirements.

Conclusion

Error Guessing alone cannot provide sufficient assurance for a safety-critical braking system. Combining BVA, State Transition Testing, risk-based testing, and systematic requirement verification improves defect detection and supports safer operation.

Question 10: Errors in Control Flow Graph Construction for Nested If Statements

1. Introduction

A Control Flow Graph (CFG) represents the possible execution paths of a program. If the else branches of nested if statements are omitted, the graph becomes incomplete. The test suite may execute normal statements without testing error-handling paths.

2. Errors in the CFG

- Missing else branches and their statements.
- Incomplete representation of decision outcomes.
- Incorrect identification of independent paths.
- Missing test cases for invalid inputs and error conditions.
- Incorrect claims of branch coverage based only on statement coverage.

3. Root Causes

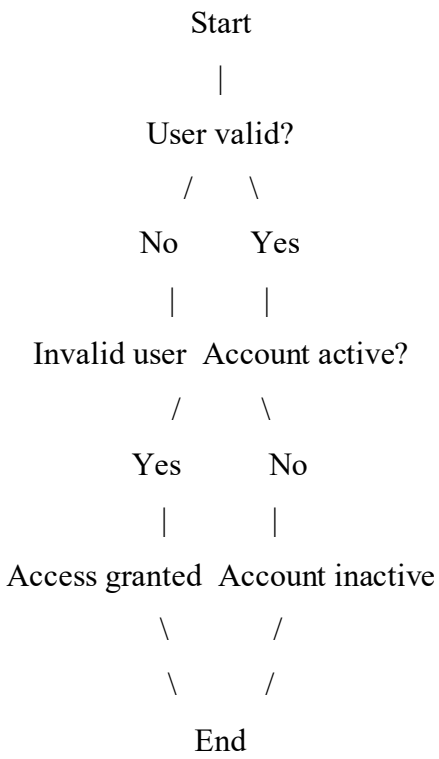
These errors can result from analyzing only the successful execution path, misunderstanding CFG construction, or failing to review all conditional statements in the source code.

4. Corrected Example

Consider this illustrative code:

if user_valid: if account_active: print("Access granted") else: print("Account inactive")else: print("Invalid user")

The corrected conceptual CFG is:



5. Independent Execution Paths

- Path 1: User valid → Account active → Access granted.

- Path 2: User valid → Account inactive → Account inactive message.
- Path 3: User invalid → Invalid user message.

6. Corrected Test Cases

Test case	User valid?	Account active?	Expected result
TC1	Yes	Yes	Access granted
TC2	Yes	No	Account inactive
TC3	No	Not evaluated	Invalid user

The third test does not need to evaluate the account status because the outer condition fails.

7. Improved Testing Strategy

1. Examine every conditional statement.
2. Include all True and False branches in the CFG.
3. Identify independent execution paths.
4. Design tests to execute each feasible path.
5. Measure statement and branch coverage separately.
6. Verify that error messages and failure handling work correctly.

Conclusion

A complete CFG must represent every decision outcome, including nested else branches. Testing all independent paths improves branch coverage and helps detect defects in both normal and error-handling logic.